Programare Python - Curs 2 2023-2024

Variabile si tipuri de date

Tipurile de date de baza sunt de doua feluri: singulare si colectii. Absolut toate variabilele în Python sunt obiecte definite în heap (malloc-ate) iar stack-ul conține doar referințe (pointeri) către acestea.

Reguli pentru denumirea variabilelor in Python

- Numele variabilelor nu pot incepe cu un numar
- Numele variabilelor nu pot contine spatii
- Numele pot contine underscore
- Nu pot fi incluse in numele variabilelor urmatoarele: : ' " , < > / ? ( ) ! @ # $ % ^ & ~ - +

Recomandări pentru denumirea variabilelor (PEP8)

- Numele variabilelor scrise cu minuscule
- De evitat utilizarea cuvintelor cu semnificatie aparte (list, str)

Tipurile de date singulare

Tipurile de date singulare sunt cele care conțin un singur element informațional

Nume	Tip	Descriere
Numere întregi (Integers)	int	Python2 avea doua tipuri de int (int si long), în python3 int este de fapt long și nu are limita de dimensiune (decat RAM-ul disponibil procesului)
Numere flotante (Floats)	float	Stocate in format mantisa + exponent cu problemele de precizie corespunzzatoare
Boolean	bool	Variabila pentru stocarea valorii de adevar (True / False)
NoneType	none	Variabilă fără tip

Variabilele nu trebuie declarate cu tipuri ca în C, Python va ghici singur ce fel de date intră într-o variabilă în funcție de conținut. Pentru a vedea ce fel de tip a asociat python unei variabile putem folosi instrucțiunea type

Variabilele nu au marcaje de dimensiune (short, long) - sunt stocate uniform

Numere intregi

```
a = 10000
b = 100.0

print(type(a))
print(type(b))
```

```
<class 'int'>
<class 'float'>
```

Incepand cu Python 3.5 se pot adauga asa numitele "type hints" care sunt recomandari (ca un fel de comentarii speciale) de care pot tine cont mediile de programare sau alte instrumente software dar pe care compilatorul Python nu le forteaza.

```
age: int =1
avg: float = 45.32
```

Pentru ca orice este un obiect, un numar intreg este o instanta a unei clase int iar instanta permite introspectia metodelor claselor. Aceasta se face cu ajutorul instructiunii **dir**

```
print(dir(a))
```

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__',
'__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__',
'__float__', '__floor__', '__floordiv__', '__format__', '__ge__',
'__getattribute__', '__getnewargs__', '__getstate__', '__gt__',
'__hash__', '__index__', '__init__', '__init_subclass__', '__int__',
'__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__',
'__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__',
'__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__',
'__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__',
'__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__',
'__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__',
'__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__',
'__xor__', 'as_integer_ratio', 'bit_count', 'bit_length', 'conjugate',
'denominator', 'from_bytes', 'imag', 'is_integer', 'numerator',
'real', 'to_bytes']
```

Metodele care conțin underscore (numite "dunder methods" de la "double under" sau "doble underscore") în nume sunt metode care permit unei instante a unui obiect sa acceseze functionalitati de baza ale limbajului.

Toate aceste metode pot fi utilizate in relatie cu numarul intreg stocat in variabila respectiva. Câteva exemple:

to_bytes - returnează un array de octeti care reprezinta intregul.

bit_length - returnează numărul de biti necesari pentru a stoca numărul respectiv în binar fără semn și zerouri preliminare

bit_count - returneaza numarul de biti setati (1)

__sizeof__ - dimensiunea în octeți în memorie a obiectului

Multe dintre metodele de tip dunder sunt utilizate în operațiile comune cu date (__lt__ și __gt__ la comparatii, __add__ - adaugare, etc.)

```
print(a.to_bytes(2, 'little'))
print(a.bit_length())

print(a.bit_count())
```

```
b"\x10'"
14
5
```

O parte dintre metodele de clasa sunt accesibile ca functii. De exemplu, metodele **abs** sau **float** corespund functiilor abs() si float()

```
nnm = -200


print(dir(nnm))

print(nnm.__abs__())
print(abs(nnm))

print(float(nnm))
print(nnm.__float__())
```

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__',
'__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__',
'__float__', '__floor__', '__floordiv__', '__format__', '__ge__',
'__getattribute__', '__getnewargs__', '__getstate__', '__gt__',
'__hash__', '__index__', '__init__', '__init_subclass__', '__int__',
'__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__',
'__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__',
'__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__',
'__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__',
'__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__',
'__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__',
'__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__',
'__xor__', 'as_integer_ratio', 'bit_count', 'bit_length', 'conjugate',
'denominator', 'from_bytes', 'imag', 'is_integer', 'numerator',
'real', 'to_bytes']
200
200
-200.0
-200.0
```

Variabilele pot contine numere oricat de mari, singura limita este data de memoria procesului python. Memoria ocupata de un numar intreg (sau float) este dificil de evaluat pentru ca impreuna cu valoarea propriu-zisa variabila va stoca multe date suplimentare.

Alocarea memoriei este automata

```
import sys

intvar =
659943549123443234453443453453453453454564564345643563456345645363
4563456345664536534563456635436456345634563456345634563456345634563456
3456345634563456345634534563456345634563456

print('Nr cifre: ' + str(len(str(intvar))))
print('Nr biti: ' + str(intvar.bit_length()))

print("Dim. raportata de sizeof: " + str(intvar.__sizeof__()))

print("Dim. raportata de compilator: " + str(sys.getsizeof(intvar)))

print(intvar * 12)



Nr cifre: 179
Nr biti: 595
Dim. raportata de sizeof: 104
Dim. raportata de compilator: 104
7919322589481318813441321441441441441454774772147722761476147614774436
1476147614797443841476147962523747614761476147614761476147614761476147
6147614761476147614761441476147614761472
```

În CPython, toate valorile de tip int sunt obiecte dinamice de tip PyLong. Nu există limită fixă de 32 sau 64 de biți; mărimea este limitată doar de memoria disponibilă.

Un int este stocat ca:

- un antet de obiect (informații despre tip și referințe),
- un câmp care indică semnul și numărul de segmente,
- un tablou de segmente numerice.

Fiecare segment conține 30 de biți utili (pe arhitecturi uzuale 64-bit). Valoarea totală a numărului este reconstruită ca sumă de segmente înmulțite cu puteri succesive ale lui 2^30.

Ordinea segmentelor este de la partea cea mai puțin semnificativă către cea mai semnificativă.

Numerele mici folosesc un singur segment. Numerele mai mari folosesc mai multe segmente. Nu există o limită fixă a valorii maxime sau minime; limita este memoria disponibilă.

Putem afla numarul de biti in care se stocheaza un segment se poate interoga folosind `sys.int_info.bits_per_digit`

```
intvar = 659943549123443234

k = sys.int_info.bits_per_digit          # de obicei 30 pe 64-bit

base = 1 << k                            # 2^k
mask = base - 1                          # (2^k - 1)

segmente = []
x = intvar                               # copiem intvar in x
while x:
    segmente.append(x & mask)              # ia ultimii k biți si ii
adauga la lista digits
    x >>= k                              # aruncă ultimii k biți

print("Biti pe digit: =", k)
print("Baza: =", bin(base))
print("nr segmente interne =", len(segmente))
print("segmente (cel mai putin semnificativ la inceput) =")
print(segmente)

# verificare: reconstruim numărul din segmente
reconstructed = 0
for i, d in enumerate(segmente):
    reconstructed += d * (base ** i)

print("reconstrucție OK?", reconstructed == intvar)
```

```
Biti pe digit: = 30
Baza: = 0b1000000000000000000000000000000
nr segmente interne = 2
segmente (cel mai putin semnificativ la inceput) =
[290470434, 614620325]
reconstrucție OK? True
```

Pentru calcule aritmetice, Python face o serie de exceptii de la tipizare, int poate fi convertit automat la float daca este nevoie

```
dd = 0

print(dd, type(dd))

dd += 0.1

print(dd, type(dd))

dd -= 0.1

print(dd, type(dd))
```

```
0 <class 'int'>
0.1 <class 'float'>
0.0 <class 'float'>
```

Numere flotante

Numerele flotante in python au aceleași probleme de reprezentare și precizie pe care le cunoaștem de la C. În codul care urmeaza, 1.1 obținut dintr-o operație aritmetică de scădere nu este același cu cel scris direct.

```
fa = 1.1
ma = 3.3 - 2.2

if fa == ma:
  print("egal")
else:
  print("diferit")

print('fa: ', fa)
print('ma: ', ma)
```

```
diferit
fa:  1.1
ma:  1.0999999999999996
```

Pentru comparatii se foloseste rotunjirea cu ajutorul functiei round()

```
if 0.1 + 0.2 + 0.3 == 0.6:
  print('egal')
else:
  print('diferit')

print(str(0.1 + 0.2 + 0.3) + ' <-> ' + str(0.6))

if round(0.1 + 0.2 + 0.3, 5) == round(0.6,5):
  print('egal')
else:
  print('diferit')

print ('round(0.1 + 0.2 + 0.3, 5) = ', round(0.1 + 0.2 + 0.3, 5))
print ('round(0.6,5) =',  round(0.6,5))
```

```
diferit
0.6000000000000001 <-> 0.6
egal
round(0.1 + 0.2 + 0.3, 5) =  0.6
round(0.6,5) = 0.6
```

Sau math.isclose()

```
import math

if math.isclose(0.1 + 0.2 + 0.3, 0.6, rel_tol=0.0, abs_tol=1e-5):
    print("egal (isclose)")
else:
    print("inegal (isclose)")

print("abs diff =", abs(0.1 + 0.2 + 0.3 - 0.6))
print("abs_tol  =", 1e-5)
```

```
egal (isclose)
abs diff = 1.1102230246251565e-16
abs_tol  = 1e-05
```

Pentru cazul in care se doresc alte tipuri de rotunjiri (floor/ceil) se pot folosi functiile corespunzatoare din modulul math. Acestea retuneaza de fiecare data numere intregi

```
import math

print('floor(1.4)', math.floor(1.4))
print('ceil(1.4)', math.ceil(1.4))

print('floor(1.6)', math.floor(1.6))
print('floor(1.6)', math.ceil(1.6))

print('floor(1.5)', math.floor(1.5))
print('floor(1.5)', math.ceil(1.5))
```

```
floor(1.4) 1
ceil(1.4) 2
floor(1.6) 1
floor(1.6) 2
floor(1.5) 1
floor(1.5) 2
```

Exista situatii in care putem dori sa obtinem efectul rotunjirii in sus sau in jos, controlabil, la variabile care contin zecimale. Functia round() rootunjeste in sus sau in jos in functie de zecimale. Ceil si floor returneaza numere intregi. Putem scrie functii care "muta virgula" inainte si inapoi astfel incat sa putem utiliza functiile descrise mai sus (floor si ceil)

```
def round_down(n, decimals=0):
    #calculeaza numarul cu care trebuie sa inmultim argumentul initial
pentru a muta virgula la dreapta cu "decimals" zecimale
    multiplier = 10**decimals
    # aplica functia floor asupra numarului obtinut in urma mutarii
virgulei, dupa care imparte la multiplicator pentru a muta virgula
inapoi
    return math.floor(n * multiplier) / multiplier
```

```
def round_up(n, decimals=0):
    multiplier = 10**decimals
    return math.ceil(n * multiplier) / multiplier

print('down: ',  round_down(1.2325, 2))
print('up: ',  round_up(1.2325, 2))


print('--------------------')
print(round(1.2325, 2))
```

```
down:  1.23
up:  1.24
--------------------
1.23
```

Operațiile aritmetice sunt cele cunoscute

Operator	Nume	Exemplu
+	Adunare	x + y
-	Scădere	x - y
*	Înmulțire	x * y
/	Împărțire	x / y
%	Modulo	x % y
**	Ridicare la putere	x ** y
//	Împărțire întreagă (stil C)	x // y

```
v1 = 10
v2 = 21
v3 = v1 + v2 #adunare
v4 = v2 - v1 #scadere
v5 = v2/v1 #impărțire
v6 = v2 // v1 # floor division

print ('v1 :' ,v1)
print ('v1 :' ,v2)
print ('v3 :' , v3)
print ('v4 :' , v4)
print ('v5 :' , v5)
print ('v6 :' , v6)
```

```
v1 : 10
v1 : 21
v3 : 31
v4 : 11
v5 : 2.1
v6 : 2
```

```
va1 = 12
va2 = 4

va3 = 12 / 4
va3 = 12 // 4


print(type(va3))
print(type(int(va3)))


<class 'int'>
<class 'int'>
```

Variabile si referinte

Variabilele in Python sunt implementate foarte diferit de C. In C, o variabila este un nume dat unei pozitii in memorie cu o adresa unica.

In Python, toate datele sunt obiecte. Acestea sunt stocate in memorie iar o variabila este pur si simplu o eticheta asociata acesteia.

Pe langa denumirea pusa de programator, Python va asocia un id fiecarui obiect pe care il stocheaza in memorie.

```
zz = 10
print(zz)
print(id(zz))

10
11654664
```

In cazul declararii unei variabile prin egalitate cu o variabila deja existenta, referinta noii variabile va fi aceeasi cu referinta variabile anterioare

```
dd = zz
print(dd)
print(id(dd))
print(id(10))

10
11654664
11654664
```

In cazul in care se aplica o operatie aritmetica asura unei variabile definite, aceasta conduce la schimbarea referintei acesteia

```
dd = dd + 1
print(dd)
```

```
print(id(dd))
print(id(11))
```

```
11
11654696
11654696
```

Referintele catre numerele relativ mici sunt pastrate intr-un cache. La aplicarea unor operatii aritmetice, daca variabila revine la o valuare antrerioara, referinta va reveni la referinta anterioara. Aceasta se intampla doar la numerele mici iaar mecanismul se numeste "interning" (internalizare)

```
fa = 5
fb = fa
fc = fa
fd = fa
print ('fa = ' + str(id(fa)))
print ('fb = ' + str(id(fb)))
print ('fc = ' + str(id(fc)))
print ('fd = ' + str(id(fd)))
fc = 13
print('-----fc = 13------')
print ('fa = ' + str(id(fa)))
print ('fb = ' + str(id(fb)))
print ('fc = ' + str(id(fc)))
print ('fd = ' + str(id(fd)))
fd = fc
print('-----fd = fc------')
print ('fa = ' + str(id(fa)))
print ('fb = ' + str(id(fb)))
print ('fc = ' + str(id(fc)))
print ('fd = ' + str(id(fd)))
fd = 13
print('-----fd = 13------')
print ('fa = ' + str(id(fa)))
print ('fb = ' + str(id(fb)))
print ('fc = ' + str(id(fc)))
print ('fd = ' + str(id(fd)))
fd = 5
print('-----fd = 5------')
print ('fa = ' + str(id(fa)))
print ('fb = ' + str(id(fb)))
print ('fc = ' + str(id(fc)))
print ('fd = ' + str(id(fd)))
```

```
fa = 11654504
fb = 11654504
fc = 11654504
fd = 11654504
```

```
-----fc = 13------
fa = 11654504
fb = 11654504
fc = 11654760
fd = 11654504
-----fd = fc------
fa = 11654504
fb = 11654504
fc = 11654760
fd = 11654760
-----fd = 13------
fa = 11654504
fb = 11654504
fc = 11654760
fd = 11654760
-----fd = 5------
fa = 11654504
fb = 11654504
fc = 11654760
fd = 11654504
```

In cazul numerelor mai mari, referinta nu mai revine la aceeasi valoare, chiar daca valoarea stocata in variabila o face. Acesta este comportamentul normal, mecanismul de interning functioneaza ca o optimizare aplicata in cazurile in care sunt sanse mari ca acele valori sa apara de foarte multe ori in cursul executiei programului

```
ffa = 5555554334
ffb = ffa
ffc = ffa
ffd = ffa
print ('ffa = ' + str(id(ffa)))
print ('ffb = ' + str(id(ffb)))
print ('ffc = ' + str(id(ffc)))
print ('ffd = ' + str(id(ffd)))
ffc = 132345433
print('-----fc = 132345433------')
print ('ffa = ' + str(id(ffa)))
print ('ffb = ' + str(id(ffb)))
print ('ffc = ' + str(id(ffc)))
print ('ffd = ' + str(id(ffd)))
ffd = ffc
print('-----fd = fc------')
print ('ffa = ' + str(id(ffa)))
print ('ffb = ' + str(id(ffb)))
print ('ffc = ' + str(id(ffc)))
print ('ffd = ' + str(id(ffd)))
ffd = 132345433
print('-----fd = 132345433------')
print ('ffa = ' + str(id(ffa)))
```

```
print ('ffb = ' + str(id(ffb)))
print ('ffc = ' + str(id(ffc)))
print ('ffd = ' + str(id(ffd)))
ffd = 5555554334
print('-----ffd = 5555554334------')
print ('ffa = ' + str(id(ffa)))
print ('ffb = ' + str(id(ffb)))
print ('ffc = ' + str(id(ffc)))
print ('ffd = ' + str(id(ffd)))
```

```
ffa = 136842707634672
ffb = 136842707634672
ffc = 136842707634672
ffd = 136842707634672
-----fc = 132345433------
ffa = 136842707634672
ffb = 136842707634672
ffc = 136842707634576
ffd = 136842707634672
-----fd = fc------
ffa = 136842707634672
ffb = 136842707634672
ffc = 136842707634576
ffd = 136842707634576
-----fd = 132345433------
ffa = 136842707634672
ffb = 136842707634672
ffc = 136842707634576
ffd = 136842707634032
-----ffd = 5555554334------
ffa = 136842707634672
ffb = 136842707634672
ffc = 136842707634576
ffd = 136842707633168
```

Utilizand functia **getrefcount** a modulului **sys** putem afla cate referinte are fiecare obiect in parte. Dar, pentru numerele intregi mici (mai mici de 256) getrefcount va returna numarul maxim 4294967295 (2**32 - 1). Acest comportament este recent (python 12) si a aparut pentru ca aceste numere sa nu fie niciodata eliminate din memorie de catre garbage collector (vezi si https://peps.python.org/pep-0683/)

```
import sys
print(sys.getrefcount(fa))
print(sys.getrefcount(ffd))
```

```
4294967295
2
```

```
for i in range(10):
  print("Numar: "+ str(i) + "numarul de referinte:" +
str(sys.getrefcount(i)))
```

```
Numar: 0numarul de referinte:4294967295
Numar: 1numarul de referinte:4294967295
Numar: 2numarul de referinte:4294967295
Numar: 3numarul de referinte:4294967295
Numar: 4numarul de referinte:4294967295
Numar: 5numarul de referinte:4294967295
Numar: 6numarul de referinte:4294967295
Numar: 7numarul de referinte:4294967295
Numar: 8numarul de referinte:4294967295
Numar: 9numarul de referinte:4294967295
```

Pentru a observa functionarea referintelor si a mecanismului de "garbage collection", urmariti urmatorul bloc de cod, care genereaza cateva numere aleatoare intregi dincolo de cele din cache care sunt acum nemuritoare, genereaza (folosind o lista de liste) o serie variabila de referinte pentru fiecare dintre ele, afiseaza numarul de referinte, apoi goleste lista si afiseaza din nou numarul de referinte.

```
import sys, random

# generam cateva numere intregi aleator
nums = [random.randint(1000, 1100) for _ in range(12)]

#initializam o lista care sa mentina un numar variabil de referinte
lista_generala = []

#in interiorul listei punem cate o sub-lista cu un numar variabil de
referinte pentru fiecare
#dintre numerele generate
for x in nums:
    k = random.randint(1, 10)
    refs = [x] * k
    lista_generala.append(refs)
    print(x, "->", k, "refs:", refs)


print("\nNumarul de referinte:")
for x in nums:
    print("Numar:", x, "numarul de referinte:", sys.getrefcount(x))


# golim lista de toate elementele sale
lista_generala.clear()


print("\nNumarul de referinte dupa stergerea listei:")
```

```python
for x in nums:
    print("Numar:", x, "numarul de referinte:", sys.getrefcount(x))
```

```
1067 -> 3 refs: [1067, 1067, 1067]
1007 -> 5 refs: [1007, 1007, 1007, 1007, 1007]
1087 -> 5 refs: [1087, 1087, 1087, 1087, 1087]
1015 -> 10 refs: [1015, 1015, 1015, 1015, 1015, 1015, 1015, 1015,
1015, 1015]
1090 -> 1 refs: [1090]
1037 -> 1 refs: [1037]
1064 -> 6 refs: [1064, 1064, 1064, 1064, 1064, 1064]
1047 -> 3 refs: [1047, 1047, 1047]
1060 -> 2 refs: [1060, 1060]
1087 -> 2 refs: [1087, 1087]
1009 -> 8 refs: [1009, 1009, 1009, 1009, 1009, 1009, 1009, 1009]
1025 -> 2 refs: [1025, 1025]

Numarul de referinte:
Numar: 1067 numarul de referinte: 7
Numar: 1007 numarul de referinte: 9
Numar: 1087 numarul de referinte: 9
Numar: 1015 numarul de referinte: 14
Numar: 1090 numarul de referinte: 5
Numar: 1037 numarul de referinte: 5
Numar: 1064 numarul de referinte: 10
Numar: 1047 numarul de referinte: 7
Numar: 1060 numarul de referinte: 6
Numar: 1087 numarul de referinte: 6
Numar: 1009 numarul de referinte: 12
Numar: 1025 numarul de referinte: 6

Numarul de referinte dupa stergerea listei:
Numar: 1067 numarul de referinte: 4
Numar: 1007 numarul de referinte: 4
Numar: 1087 numarul de referinte: 4
Numar: 1015 numarul de referinte: 4
Numar: 1090 numarul de referinte: 4
Numar: 1037 numarul de referinte: 4
Numar: 1064 numarul de referinte: 4
Numar: 1047 numarul de referinte: 4
Numar: 1060 numarul de referinte: 4
Numar: 1087 numarul de referinte: 4
Numar: 1009 numarul de referinte: 4
Numar: 1025 numarul de referinte: 6
```

Devine clar astfel ca referintele nu sunt referinte catre variabile ci referinte catre valori. Acestea sunt in realitate obiectele stocate in memorie.

Tipul boolean

```
bl = True
print(bl, type(bl))
print (dir(bl))

True <class 'bool'>
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__',
'__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__',
'__float__', '__floor__', '__floordiv__', '__format__', '__ge__',
'__getattribute__', '__getnewargs__', '__getstate__', '__gt__',
'__hash__', '__index__', '__init__', '__init_subclass__', '__int__',
'__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__',
'__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__',
'__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__',
'__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__',
'__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__',
'__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__',
'__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__',
'__xor__', 'as_integer_ratio', 'bit_count', 'bit_length', 'conjugate',
'denominator', 'from_bytes', 'imag', 'is_integer', 'numerator',
'real', 'to_bytes']
```

Tipul None

None este un obiect, și face parte din clasa NoneType.

- Orice comparație între None și orice altceva care nu este None va returna False
- None nu este 0
- None nu este un șir gol
- None nu este același lucru ca False

```
mv = None

if mv is None:
    print('avem none')

avem none
```

tipul None nu poate fi utilizat in alte operatii

```
print(mv + 12)

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-166/2865146185.py in <cell line: 0>()
----> 1 print(mv + 12)

TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

Tipul text (sir)

Poate fi privit ca un tip de date care este in acelasi timp unitar si colectie.

```
ss = "Ala Bala Portocala"
print (ss)
print(ss[2])
```

```
Ala Bala Portocala
a
```

```
print(dir(ss))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
'__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__',
'__init__', '__init_subclass__', '__iter__', '__le__', '__len__',
'__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold',
'center', 'count', 'encode', 'endswith', 'expandtabs', 'find',
'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii',
'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric',
'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust',
'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix',
'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition',
'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip',
'swapcase', 'title', 'translate', 'upper', 'zfill']
```

```
print(ss.upper())
print(ss.lower())
print(ss.split())
print(ss.split('a'))
print(ss.split('al'))
```

```
ALA BALA PORTOCALA
ala bala portocala
['Ala', 'Bala', 'Portocala']
['Al', ' B', 'l', ' Portoc', 'l', '']
['Ala B', 'a Portoc', 'a']
```

```
str1 = "Ala bala Portocala"
str2 = 'Ala bala Portocala'
str3 = "Ala' bala' Porto'cala"

print(str1 + str2 + str3)
```

```
Ala bala PortocalaAla bala PortocalaAla' bala' Porto'cala
```

```
sstr1 = 'La reve"dere'
sstr2 = "La rev'edere"
```

```
print(sstr1)
print(sstr2)
```

```
La reve"dere
La rev'edere
```

Șirul însă este un tip de date secvențial ordonat (o secvență ordonată) și atunci putem utiliza indexare (accesarea unui singur element) și izolare (accesarea unei subsecvențe).

Indexarea șirurilor în Python pornește de la 0

Indexarea cu valori pozitive porneste numaratoarea din partea *stanga*

```
print(ss)
print(ss[0], ss[1], ss[2])
```

```
Ala Bala Portocala
A l a
```

Indexarea cu valori negative porneste numaratoarea din partea *dreapta*

```
print(ss)
print(ss[-1], ss[-2], ss[-3])
```

```
Ala Bala Portocala
a l a
```

Putem folosi izoarea (slicing)

```
string = 'abcdefghijklmnopqrstuvwxyz'


slice1 = string[10:]
slice2 = string[:10]     #de la caracterul 0 pana la caracterul 10
(exclusiv)
slice3 = string[4:8]     #de la caracterul 5 pana la caracterul 9
(exclusiv)
slice4 = string[4:18:3]  #de la caracterul 5 pana la caracterul al 18-
lea cu pasul 3
slice5 = string[::3]     #de la început pana la final cu pasul 3
slice6 = string[::-1]    #pasul negativ baleiaza de la dreapta la
stanga

print('slice1 ', slice1)
print('slice2 ', slice2)
print('slice3 ', slice3)
print('slice4 ', slice4)
print('slice5 ', slice5)
print('slice6 ', slice6)
```

```
slice1  klmnopqrstuvwxyz
slice2  abcdefghij
slice3  efgh
slice4  ehknq
slice5  adgjmpsvy
slice6  zyxwvutsrqponmlkjihgfedcba
```

Șirurile sunt immutabile în Python - le putem reatribui dar nu putem seta fragmente din ele. Dacă dorim să modificăm o parte dintr-un șir trebuie să rescriem întregul șir în aceeași variabilă - folosing tehnici de slicing ale zonelor pe care nu le dorim modificate.

De exemplu, pentru a modifica șirul "Sare" în "Care" trebuie modificată (la prima vedere) litera "S" in "C". Aceasta nu se poate face prin accesul la litera cu ajutorul indexului dar se poate face prin rescrierea întregului șir cu inlocuirea primei litere și păstrarea unui slice al caracterelor nemodificate.

```
name = "Sare"

print(name[0])

# name[0] = 'C'  # nu functioneaza

name = 'C' + name[1:]

print(name)

S
Care

decl = "Nimeni nu va reusi niciodata sa ma prinda"

print(len(decl))

print(decl.__len__())

41
41

print(decl.count('a'))
print(decl.find('a'))
print(decl.index('a'))

6
11
11
```

Secvente escape

O secvență escape este un grup de caractere care nu vor apărea așa cum sunt scrise ci vor fi convertite în altceva, de regulă în elemente imposibil de afișat direct (newline, tab)

Escape Sequence	Meaning
'	Single quote
\"	Double quote
\	Backslash
\n	Newline
\r	Carriage Return
\t	Horizontal Tab
\b	Backspace
\f	Formfeed
\v	Vertical Tab
\0	Null Character
\N{Name}	Unicode character Database named lookup
\uxxxxxxxx	Unicode character with a 16-bit hex value
\Uxxxxxxxx	Unicode character with a 32-bit hex value
\000	Character with octal value ooo
\xhh	Character with hex value hh

```
print('can\'t do this')
print("Cred ca e foarte \"destept\"")
```

```
can't do this
Cred ca e foarte "destept"
```

```
print('can\'t do\n this')
```

```
can't do
 this
```

```
print('can\'t do\r this')
```

```
can't do  this
```

```
print ("Vasilica e prost \b\b\b\b\b\b destept")
```

```
Vasilica e prost  destept
```

```
print ("This is a \N{nerd face}")
print ("This is a \N{Circled Latin Capital Letter N}")
```

```
This is a 
This is a ⓝ
```

Unicode

Unicode este un standard internațional pentru reprezentarea caracterelor în format numeric, astfel încât orice simbol scris - litere, cifre, semne de punctuație, caractere speciale sau emoji are un **code point** - un numar intreg, de obicei exprimat in hexazecimal, care poate avea valori intre 0 pana la 1.114.111 (10FFFF). Pentru a reprezenta toate code-pointurile din unicode sunt necesari maximum 21 de biti.

Reprezentarea in memorie a caracterelor unicode trebuie ales un pattern de biti, numit encoding. Din multe motive, exista mai multe moduri de reprezentare a code-pointurilor - utf8, utf16, utf32. Fiecare cu problemele sale.

- **utf32** stocheaza fiecare caracter pe 32 de biti. Acest format permite toate code-pointurile dar este extrem de ineficient - majoritatea textului de pe un sistem informatic obisnuit este compus din caractere ASCII. Codul escape pentru acesta este \U
- **utf8** stocheaza fiecare caracter intr-un numar variabil de biti. Caracterele ASCII de baza sunt stocate pe un singur octet (char), caracterele extinse (diacritice) sunt stocate de obicei in 2 octeti, emoji-urile si alte caractere in grafii mai neobisnuite in mai multi octeti (pana la 4). Nu exista secventa escape pentru utf8
- **utf16** aparut initial ca encoding de lungime fixa, cand se credea ca 65536 code-points sunt suficiente pentru reprezentarea tuturor caracterelor existente. Odata cu introducerea caracterelor asiatice nu a mai fost suficient dar utf16 fusese deja adoptat in windows si java asa ca a fost si el convertit in encoding variabil. Secventa escape este \u dar nu poate prelucra decat spatiul de baza al utf16 (unul dintre cele 65536 caractere de baza). \u nu suporta caracterele unicode apartinand code-pointurilor superioare.

Exista online baza de date cu aceste caractere, care afiseaza toate modurile de identificare a acestor caractere, denumire, variate tipuri de encoding, etc. https://unicode-table.com/en/24C3/

De exemplu, caracterul Đ poate fi reprezentat folosind secvente escape astfel:

```
print ("Escape folosind nume: \N{Latin Capital Letter D with Stroke}")
print ("Escape folosind utf32: \U00000110")
print ("Escape folosind utf16: \u0110")
```

```
Escape folosind nume: Đ
Escape folosind utf32: Đ
Escape folosind utf16: Đ
```

secventele escape \000 si \xhh pot fi folosite doar pentru caracterele ASCII (0-255)

```
print("caracterul a se poate scrie si: \141")   # 'a'  (octal 141 =
decimal 97)
print("caracterul a se poate scrie si: \x61")   # 'a'  (hex 61 =
decimal 97)
```

```
caracterul a se poate scrie si: a
caracterul a se poate scrie si: a
```